COS 730 – Assignment 2

From Behavioural Models to Optimised Implementation

Module	COS 730 – Software Engineering (I)
Assignment	2
Student Name	Yohali Malaika Kamangu
Student Number	u23618583
Date Submitted	14 May 2026
GitHub Repository	https://github.com/YourfavCompSciGirlie/COS730
Language	Java 17 (Maven)

University of Pretoria
Department of Computer Science

Contents

1 Introduction

This report documents the analysis, redesign, and empirical evaluation of the **Intelligent Submission and Review System (ISRS)** — a system responsible for processing research output submissions, assigning peer reviewers, evaluating scores, and notifying researchers of outcomes. The work proceeds through six tasks: implementing a baseline system directly from a provided sequence diagram, identifying design flaws, extracting decision logic into a decision table, producing an optimised sequence diagram, implementing the optimised design, and empirically comparing the two versions.

The system was implemented in **Java 17** using **Maven** as the build tool. Both implementations (baseline and optimised) are available in the public GitHub repository linked on the title page, organised in `Original/` and `Optimised/` directories respectively. No external frameworks were used — the implementations rely on plain Java to keep the focus on design principles and architectural patterns.

Running the code: Executable JAR files are provided for convenience. From the repository root:

```
# Baseline demo
java -jar Original/target/original-1.0-SNAPSHOT.jar

# Baseline benchmark
java -cp Original/target/original-1.0-SNAPSHOT.jar com.isrs.original.
    Benchmark

# Optimised demo
java -jar Optimised/target/optimised-1.0-SNAPSHOT.jar

# Optimised benchmark
java -cp Optimised/target/optimised-1.0-SNAPSHOT.jar com.isrs.optimised.
    Benchmark
```

2 Task 1 – Baseline Implementation

2.1 Implementation Overview

The baseline implementation faithfully reproduces the provided sequence diagram. Every participant, message call, self-call, loop, and alternative fragment in the diagram has a direct counterpart in the Java code, with inline comments mapping each method invocation to the corresponding diagram step.

Source code: `Original/src/main/java/com/isrs/original/`

2.2 Participant-to-Class Mapping

Table 1: Mapping of sequence diagram participants to Java classes

Diagram Participant	**Java Class**	**Role**
Researcher (actor)	—	Simulated in `Main.java`
UI	`UI.java`	Boundary — receives submission, delegates to controller
SubmissionController	`SubmissionController.java`	Fat controller — orchestrates entire pipeline
Validator	`Validator.java`	Validates submission format
Database	`Database.java`	Monolithic persistence (submissions, reviewers, scores)
ReviewerManager	`ReviewerManager.java`	Filters reviewers by conflict and workload
Reviewer	`Reviewer.java`	Domain entity — assigned to review, submits scores
EvaluationManager	`EvaluationManager.java`	Manages evaluation: scoring, consensus, rules
NotificationService	`NotificationService.java`	Sends outcome notifications

2.3 Message Call Traceability

Table 2 maps every message call from the sequence diagram to its Java implementation.

Table 2: Sequence diagram message call traceability

#	**Diagram Message**	**Java Method Call**
1	Researcher → UI : submitResearchOutput(data)	`UI.submitResearchOutput()`
2	UI → SubmissionController : submit(data)	`SubmissionController.submit()`
3	Controller → Validator : validateFormat(data)	`Validator.validateFormat()`
4	Controller → Database : saveSubmission(data)	`Database.saveSubmission()`
5	Controller → ReviewerManager : getAvailableReviewers()	`ReviewerManager.getAvailableReviewers()`
6	ReviewerManager → Database : fetchReviewers()	`Database.fetchReviewers()`
7	ReviewerManager → self : filterConflicts()	`ReviewerManager.filterConflicts()`
8	ReviewerManager → self : checkWorkload()	`ReviewerManager.checkWorkload()`
9	loop: Controller → Reviewer : assignReview()	`Reviewer.assignReview()`

#	Diagram Message	Java Method Call
10	Controller → EvalManager : startEvaluation()	`EvaluationManager.startEvaluation()`
11	loop: Reviewer → EvalManager : submitScore()	`Reviewer.submitScore()`
12	EvalManager → Database : saveScore(score)	`Database.saveScore()`
13	EvalManager → self : calculateAverage()	`EvaluationManager.calculateAverage()`
14	EvalManager → self : checkConsensus()	`EvaluationManager.checkConsensus()`
15	EvalManager → self : applyRules()	`EvaluationManager.applyRules()`
16	alt: EvalManager → NotifService : notifyX()	`NotificationService.notifyX()`
17	NotificationService → Researcher : sendNotification()	`NotificationService.sendNotification()`

2.4 Execution Output

The baseline produces the following output for a valid submission with 5 reviewers (3 eligible after filtering):

```
[UI] Research output received: Deep Learning for Climate Prediction
[SubmissionController] Processing submission...
[Validator] Validation PASSED
[Database] Submission saved with ID: SUB-XXXXXXXX
[ReviewerManager] Getting available reviewers...
[Database] Fetching all reviewers... (5 found)
[ReviewerManager] Removed 1 reviewer(s) with conflicts
[ReviewerManager] Removed 1 overloaded reviewer(s)
[ReviewerManager] Returning 3 eligible reviewers
[EvaluationManager] Evaluation started
[EvaluationManager] Average score: 8.00
[EvaluationManager] Consensus check: REACHED (range: 1.0)
[EvaluationManager] Rules applied. Outcome: ACCEPTED
[NotificationService] Preparing ACCEPTANCE notification
[NotificationService] Notification sent to researcher
```

Scores of 7.5, 8.0, and 8.5 yield an average of 8.0 with consensus reached (range $1.0 \leq 3.0$), resulting in acceptance.

3 Task 2 – Design Analysis

The baseline sequence diagram contains several design flaws that affect maintainability, cohesion, coupling, and extensibility. This section identifies and analyses each issue.

3.1 Redundant Message Calls

Issue 1: Per-Score Database Persistence (3 redundant calls). In the baseline, each reviewer's score triggers an individual `Database.saveScore(score)` call inside a loop. This results in **3 separate database interactions** for what is logically a single batch operation. Each call carries the overhead of method invocation, HashMap access, and console logging. A single `saveAll(List<Score>)` batch call would be semantically clearer and more efficient.

```
loop [each reviewer]:
    Reviewer -> EvaluationManager : submitScore(score)
    EvaluationManager -> Database : saveScore(score)    // called 3
   times
```

Issue 2: Three Separate Notification Methods. The notification phase uses three distinct methods (`notifyAcceptance()`, `notifyRejection()`, `notifyRevision()`) instead of a single parameterised `notify(outcome)` method. This violates the DRY principle and makes adding new outcome types require creating a new method plus updating every call site.

Issue 3: Separate filterConflicts() and checkWorkload() Self-Calls. The `ReviewerManager` makes two self-calls that iterate over the reviewer list sequentially. These could be combined into a single filtering pass using a composed predicate (e.g., `!hasConflict && workload < 5`), reducing both the number of message calls and the number of list iterations.

3.2 GRASP Principle Violations

Controller Bloat (High Cohesion Violation). `SubmissionController` is a fat controller responsible for:

1. Validating submission format (step 3)
2. Persisting the submission to the database (step 5)
3. Requesting filtered reviewers from ReviewerManager (step 6)
4. Assigning reviews in a loop (step 9)
5. Triggering the evaluation lifecycle (step 10)

This violates the **Controller** GRASP principle, which states that a controller should delegate to service objects rather than containing business logic.

Creator Violation. The `SubmissionController` directly calls `Reviewer.assignReview()` inside a loop. According to the **Creator** principle, responsibility for creating review assignments should belong to the object that aggregates or closely uses the Reviewer objects — a `ReviewAssignmentService`, not the controller.

Information Expert Violation. `ReviewerManager` accesses `Database.fetchReviewers()` directly, while `SubmissionController` accesses `Database.saveSubmission()` directly. There is no consistent data access pattern. According to Information Expert, persistence operations should be handled by dedicated repository objects, not spread across multiple classes.

3.3 High Coupling / Low Cohesion

SubmissionController Coupling. `SubmissionController` has 4 constructor dependencies (`Validator`, `Database`, `ReviewerManager`, `EvaluationManager`), yielding an efferent coupling (C_e) of **4**. Meanwhile, `EvaluationManager` depends on both `Database` and `NotificationService`, combining evaluation logic with notification dispatch.

Database as God Object. The `Database` class manages three unrelated concerns: submission persistence, reviewer data access, and score persistence. This violates the Single Responsibility Principle. Each concern should be in its own repository class.

EvaluationManager Mixed Responsibilities. `EvaluationManager` combines four distinct responsibilities via self-calls: receiving scores, computing statistics, checking consensus, and applying business rules. These should be separated so each can be modified independently.

3.4 Poor Decision Logic Handling

The acceptance/rejection/revision decision is buried inside `EvaluationManager.applyRules()`:

```
if (averageScore >= 7.0 && consensusReached) {
    result = "accepted";
} else if (averageScore < 5.0) {
    result = "rejected";
} else {
    result = "revision";
}
```

This logic is **opaque** (conditions are not documented), **string-typed** (inviting typos), **tightly coupled** (changing a rule requires modifying EvaluationManager), and **untestable in isolation**. The decision logic should be extracted into a dedicated `DecisionEngine` — a **Pure Fabrication** in GRASP terms — so that rules can be maintained, tested, and extended independently of the evaluation workflow.

3.5 Inefficient Control Flow

Reviewer assignment loop controlled externally. The `SubmissionController` iterates over eligible reviewers and calls `Reviewer.assignReview()` individually. This loop should be delegated to a dedicated `ReviewAssignmentService` that encapsulates the assignment strategy, following the **Indirection** GRASP principle. The current design means that any change to the assignment strategy (e.g., limiting the number of assigned reviewers, or selecting by expertise match) requires modifying the controller.

Score submission loop creates bidirectional coupling. In the scoring loop, the controller iterates over reviewers and calls `Reviewer.submitScore(evaluationManager)`, which in turn calls `EvaluationManager.submitScore(score)` and then `Database.saveScore(score)`. This creates a chain of three method calls per reviewer where the `Reviewer` entity depends on the `EvaluationManager` service. A cleaner flow would have the service collect scores from reviewers without the reviewer needing to know about the evaluation subsystem.

Sequential self-calls in EvaluationManager. The sequence `calculateAverage()` → `checkConsensus()` → `applyRules()` is a chain of three self-calls that must be invoked

in the correct order, with each depending on state set by the previous call. This is fragile: calling `applyRules()` before `calculateAverage()` would produce incorrect results. A single `evaluate(scores)` method would eliminate this temporal coupling and reduce the number of interactions in the sequence diagram.

3.6 Behavioural Modelling Quality

Several issues in the baseline reflect poor behavioural modelling practices in the sequence diagram itself:

1. **Ambiguous activation lifelines:** The diagram does not make clear when control is held by `SubmissionController` versus when it is delegated. The controller's lifeline spans the entire interaction, suggesting it owns all behaviour — a modelling smell that corresponds to the fat controller anti-pattern.
2. **Missing return messages:** Several interactions lack explicit return messages (e.g., `ReviewerManager` → `SubmissionController : filteredReviewers` is present, but `Reviewer.assignReview()` has no return). This makes the diagram inconsistent and harder to trace.
3. **Self-calls obscure responsibility:** The five self-calls in the diagram (`filterConflicts`, `checkWorkload`, `calculateAverage`, `checkConsensus`, `applyRules`) obscure the internal logic that should be modelled as separate interactions with dedicated objects. Self-calls in sequence diagrams should be reserved for genuine recursive or internal-state operations, not for multi-step algorithms that warrant their own participants.
4. **No guard conditions on alternatives:** The alt fragments for notification dispatch (`accepted`/`rejected`/`revision`) lack formal guard conditions (e.g., `[avg >= 7.0 AND consensus]`). This makes the diagram incomplete as a behavioural specification — the reader cannot determine what triggers each path without inspecting the code.
5. **Reviewer-EvaluationManager coupling in the diagram:** The diagram shows `Reviewer` → `EvaluationManager : submitScore(score)`, which implies that the domain entity `Reviewer` initiates communication with the service `EvaluationManager`. This violates the principle that entities should not depend on control objects, and results in a diagram that is harder to refactor.

3.7 Summary of Design Issues

Table 3: Summary of identified design issues in the baseline

Issue	Principle Violated	Impact
Fat SubmissionController	Controller, High Cohesion	Hard to test, maintain, extend
Monolithic Database	SRP, Information Expert	Changes to one concern affect all three
3 notification methods	DRY	Adding outcomes requires new methods
Per-score DB saves in loop	Efficiency	3 calls instead of 1 batch
Opaque decision logic	Pure Fabrication	Rules untestable in isolation
EvaluationManager self-calls	High Cohesion	Temporal coupling between 3 methods
Reviewer → EvalManager coupling	Low Coupling	Entity depends on service
External assignment loop	Indirection	Assignment strategy locked in controller
Ambiguous diagram lifelines	Behavioural modelling	Fat controller smell not visible
Missing guard conditions	Behavioural completeness	Alt paths unspecified in diagram

4 Task 3 – Decision Table

4.1 Extracted Decision Logic

All decision points in the system were identified and extracted into a single decision table. The conditions are evaluated in order: format validation first (since invalid submissions short-circuit the pipeline), then average score and consensus.

4.2 Decision Table

Table 4: Decision table for the ISRS evaluation logic

Rule	C1: Valid?	C2: Avg $\geq$ 7.0?	C3: Consensus?	C4: Avg $<$ 5.0?	Action
R1	N	—	—	—	**REJECT** (validation)
R2	Y	Y	Y	N	**ACCEPT**
R3	Y	—	—	Y	**REJECT**
R4	Y	Y	N	N	**REVISION REQUIRED**
R5	Y	N	—	N	**REVISION REQUIRED**

4.3 Condition Definitions

Table 5: Condition definitions and their sources in the baseline

Condition	Definition	Baseline Source
C1: Format Valid?	Title, authors, field, and abstract are all non-null and non-empty	`Validator.validateFormat()`
C2: Avg $\geq$ 7.0?	Arithmetic mean of all reviewer scores meets the acceptance threshold	`EvalManager.calculateAverage()`
C3: Consensus?	Range (max $-$ min) of reviewer scores is ≤ 3.0	`EvalManager.checkConsensus()`
C4: Avg $<$ 5.0?	Average score falls below the rejection threshold	`EvalManager.applyRules()`

4.4 Completeness Verification

The table is **complete** — every combination of conditions maps to exactly one action:

- **R1 (Invalid format):** Immediately rejected regardless of score conditions. Short-circuits the pipeline before reviewer assignment.
- **R2 (Valid + avg $\geq$ 7.0 + consensus):** The only path to acceptance. Both quality (average) and agreement (consensus) are required.
- **R3 (Valid + avg $<$ 5.0):** Rejected outright. Consensus is irrelevant when the average is below the minimum threshold.
- **R4 (Valid + avg $\geq$ 7.0 + no consensus):** Reviewers disagree significantly despite a high average. Revision required.
- **R5 (Valid + 5.0 $\leq$ avg $<$ 7.0):** Average is in the indeterminate range. Regardless of consensus, revision is required.

4.5 Advantages Over Scattered Conditionals

The decision table improves on the baseline's `applyRules()` method in several ways:

1. **Transparency:** All rules are visible in a single table rather than buried in if-else chains.
2. **Maintainability:** Adding a new rule requires adding a row to the table and a corresponding condition check — no modification to existing rules.
3. **Testability:** Each rule can be tested independently by constructing the corresponding set of conditions.
4. **Traceability:** Each rule maps directly to a system requirement.

4.6 Replacing Scattered Diagram Logic

In the baseline sequence diagram, decision logic is scattered across three distinct locations:

1. **Validation alt fragment** (lines 16–18 of the diagram): An `alt invalid / else valid` fragment controls whether the submission proceeds. This corresponds to

condition C1 and rule R1 in the decision table.

2. **Self-call chain** (lines 36–38): `calculateAverage()`, `checkConsensus()`, and `applyRules()` are three sequential self-calls on `EvaluationManager` that compute conditions C2, C3, and C4 individually with no visible relationship between them. The decision table unifies these into a single lookup.
3. **Notification alt fragment** (lines 40–47): An `alt accepted / rejected / revision` fragment selects the notification method. The conditions that trigger each branch are not specified in the diagram — the reader must inspect the code. The decision table makes these conditions explicit (R2 → accepted, R3 → rejected, R4/R5 → revision).

The decision table consolidates all three locations into a single, self-contained specification. The self-calls become a single `DecisionEngine.evaluate(scores)` call, and the notification alt fragment's guard conditions become the Action column of the table.

4.7 Mapping Rules to System Behaviour

Table 6 maps each decision table rule to the concrete system behaviour it triggers, tracing the path from evaluation through notification.

Table 6: Decision table rules mapped to system behaviour

Rule	Action	System Behaviour
R1	REJECT (validation)	Pipeline short-circuits at `Validator.validateFormat()`. No reviewers assigned, no scores collected. Error returned to UI immediately.
R2	ACCEPT	Scores collected → average $\geq$ 7.0 with consensus → `notifyAcceptance()` → `sendNotification()` to researcher.
R3	REJECT	Scores collected → average $<$ 5.0 → `notifyRejection()` → `sendNotification()` to researcher. Consensus irrelevant.
R4	REVISION REQUIRED	Scores collected → average $\geq$ 7.0 but consensus not reached (range $>$ 3.0) → `notifyRevision()` → `sendNotification()` to researcher.
R5	REVISION REQUIRED	Scores collected → 5.0 $\leq$ average $<$ 7.0 → `notifyRevision()` → `sendNotification()` to researcher. Consensus check immaterial.

5 Task 4 – Optimised Sequence Diagram

5.1 Design Changes

The optimised sequence diagram addresses every design flaw identified in Task 2. The key architectural changes are:

Change 1: Split the Fat Controller. The monolithic `SubmissionController` is split into three focused components:

- `SubmissionController` — handles validation, submission persistence, and thin delegation to the services below
- `ReviewAssignmentService` — handles reviewer selection, filtering, and score collection
- `EvaluationService` — orchestrates score persistence and outcome determination

Change 2: Repository Pattern (Replace God Object). The monolithic `Database` class is replaced by three single-responsibility repositories:

- `SubmissionRepository` — submission persistence only
- `ReviewerRepository` — reviewer data access only
- `ScoreRepository` — score persistence only

Change 3: Centralised Decision Engine. A `DecisionEngine` encapsulates the decision table from Task 3. The `EvaluationService` delegates outcome determination to the `DecisionEngine`, eliminating the scattered self-calls.

Change 4: Parameterised Notifications. Three separate notification methods are replaced by a single `NotificationService.notify(EvaluationOutcome)`, where `EvaluationOutcome` is a type-safe enum (`ACCEPTED`, `REJECTED`, `REVISION_REQUIRED`).

Change 5: Batch Score Persistence. The per-score save loop (`Database.saveScore()` $\times$ N) is replaced by `ScoreRepository.saveAll(List<Score>)` — a single batch operation.

Change 6: Decoupled Reviewer Scoring. In the baseline, `Reviewer` directly calls `EvaluationManager.submitScore()`, creating tight coupling. In the optimised version, `ReviewAssignmentService` calls `Reviewer.generateScore()` (a pure function) and collects scores, eliminating the Reviewer $\rightarrow$ EvaluationManager dependency.

5.2 Optimised Participant List

Table 7: Optimised sequence diagram participants

Participant	Stereotype	Responsibility
Researcher	actor	Submits research output
UI	boundary	Receives input, returns result
SubmissionController	control	Validation + persistence + delegation
Validator	entity	Format validation
SubmissionRepository	entity	Submission CRUD
ReviewAssignmentService	control	Reviewer filtering + score collection
ReviewerRepository	entity	Reviewer data access
Reviewer	entity	Score generation
EvaluationService	control	Score persistence + evaluation delegation
ScoreRepository	entity	Score batch persistence
DecisionEngine	entity	Decision table evaluation
NotificationService	boundary	Outcome notification

5.3 Optimised Sequence Flow

```
Researcher -> UI : submitResearchOutput(data)
  UI -> SubmissionController : submit(data)
    SubmissionController -> Validator : validateFormat(data) -> isValid
    alt [invalid]: return error
    else [valid]:
      SubmissionController -> SubmissionRepository : save(data) ->
   submissionId
      SubmissionController -> ReviewAssignmentService :
          assignAndCollectScores(submissionId)
        ReviewAssignmentService -> ReviewerRepository : findAll() ->
   candidates
        [filter conflicts & workload in single pass]
        loop: ReviewAssignmentService -> Reviewer : generateScore() ->
   value
      -> List<Score>
      SubmissionController -> EvaluationService : evaluate(submissionId,
    scores)
        EvaluationService -> ScoreRepository : saveAll(scores)
        EvaluationService -> DecisionEngine : evaluate(scores) ->
   Outcome
      -> EvaluationOutcome
      SubmissionController -> NotificationService : notify(outcome)
    -> result
  -> result
```

The full PlantUML diagram is available at `Optimised/diagrams/optimised_sequence.puml`. Figure 1 shows the rendered diagram.

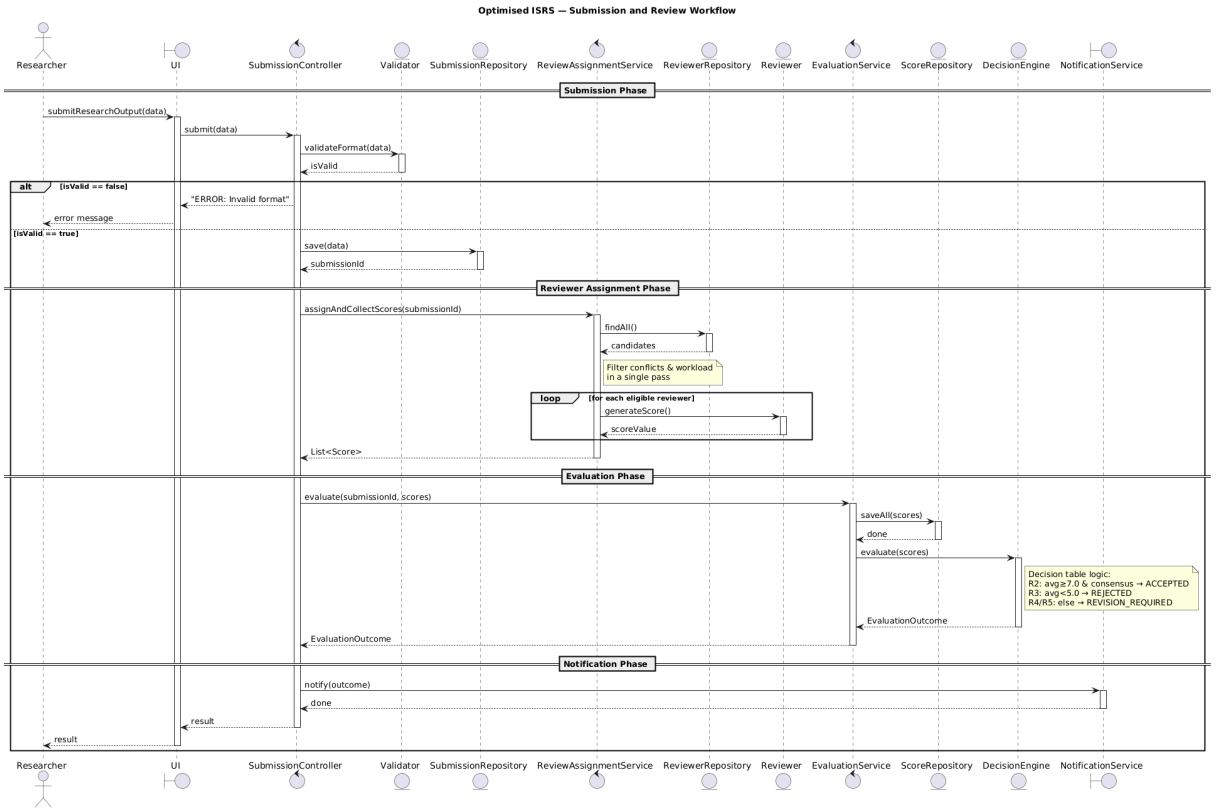

Figure 1: Optimised ISRS sequence diagram

5.4 Interaction Count Comparison

Table 8: Interaction count comparison between baseline and optimised diagrams

Metric	Baseline	Optimised
Named message calls (excl. loops)	17	10
Self-calls	5	0
Database/Repository calls	5	3
Notification methods	3 (+1 send)	1
Loop fragments	2	1
Participants	9	12

The optimised diagram has more participants (12 vs 9) but significantly fewer interactions — a trade-off of structural complexity for reduced behavioural complexity.

5.5 Design Quality Improvements Demonstrated

Improved Cohesion. Each class now has a single, well-defined responsibility. The baseline's `SubmissionController` handled validation, persistence, reviewer assignment, evaluation, and notification — five concerns. The optimised design distributes these across `SubmissionController` (validation + delegation), `ReviewAssignmentService` (reviewer filtering + score collection), `EvaluationService` (evaluation orchestration), `DecisionEngine` (decision logic), and `NotificationService` (notification). Similarly, the monolithic `Database` splits into three single-purpose repositories.

Reduced Coupling. The optimised `SubmissionController` has 5 dependencies (one more than the baseline's 4), but each is now a focused, single-responsibility service rather than a mixed-concern class. The key coupling improvement is elsewhere: the `Reviewer` entity no longer depends on `EvaluationManager` — it exposes a pure `generateScore()` method with no outgoing service dependencies. The bidirectional Reviewer ↔ EvaluationManager coupling is eliminated entirely. Additionally, the monolithic `Database` is no longer a coupling hub; each repository is used only by the component that needs it.

Cleaner Control Flow. The optimised diagram reads as a linear pipeline with clear phase separations (Submission → Assignment → Evaluation → Notification). The five self-calls in the baseline (`filterConflicts`, `checkWorkload`, `calculateAverage`, `checkConsensus`, `applyRules`) are eliminated. Filtering is internalised within `ReviewAssignmentService`, and scoring/consensus/rules are consolidated into the `DecisionEngine.evaluate()` call.

Better Separation of Concerns. The design follows a boundary–control–entity (BCE) architecture: `UI` and `NotificationService` are boundaries, the three services are controllers, and repositories, `Validator`, `Reviewer`, and `DecisionEngine` are entities. This separation ensures that changes to persistence, business rules, or notifications do not propagate across concern boundaries.

6 Task 5 – Optimised Implementation

6.1 Implementation Overview

The optimised implementation directly reflects the sequence diagram from Task 4. Every participant maps to a Java class, and every message call maps to a method invocation.

Source code: `Optimised/src/main/java/com/isrs/optimised/`

6.2 Class Mapping

Table 9: Mapping between optimised and baseline classes

Optimised Class	Baseline Equivalent	Key Change
`SubmissionController`	`SubmissionController`	95 → 65 LOC; delegates
`ReviewAssignmentService`	Controller + ReviewerManager	New: reviewer filtering + scores
`EvaluationService`	`EvaluationManager`	Delegates to DecisionEngine
`DecisionEngine`	`applyRules()`	New: decision table logic
`SubmissionRepository`	`Database` (partial)	Submission persistence only
`ReviewerRepository`	`Database` (partial)	Reviewer data access only
`ScoreRepository`	`Database` (partial)	Score persistence (batch)
`NotificationService`	`NotificationService`	53 → 19 LOC; single method
`EvaluationOutcome`	—	New: type-safe enum
`Validator`	`Validator`	Unchanged
`UI`	`UI`	Unchanged
`Reviewer`	`Reviewer`	Pure `generateScore()`

6.3 Key Refactoring Decisions

6.3.1 Reviewer Decoupling

In the baseline, `Reviewer.submitScore(EvaluationManager)` creates tight coupling. In the optimised version, `Reviewer.generateScore()` is a pure function with no dependencies:

```
// Baseline: Reviewer calls EvaluationManager (tight coupling)
reviewer.submitScore(evaluationManager);

// Optimised: Service orchestrates, Reviewer has no dependency
double score = reviewer.generateScore();
```

```
scores.add(new Score(reviewer.getReviewerId(), submissionId, score));
```

6.3.2 Batch Score Persistence

The baseline saves each score individually inside a loop. The optimised version collects all scores first, then persists them in a single batch:

```
// Baseline: N database calls
for (Reviewer r : reviewers) {
    r.submitScore(evaluationManager); // calls database.saveScore()
}

// Optimised: 1 batch call
scoreRepository.saveAll(scores);
```

6.3.3 Decision Engine Encapsulation

The `DecisionEngine.evaluate(List<Score>)` method encapsulates all decision table logic:

```
public EvaluationOutcome evaluate(List<Score> scores) {
    double average = scores.stream()
            .mapToDouble(Score::getScore).average().orElse(0.0);
    double min = scores.stream()
            .mapToDouble(Score::getScore).min().orElse(0.0);
    double max = scores.stream()
            .mapToDouble(Score::getScore).max().orElse(0.0);
    boolean consensus = (max - min) <= CONSENSUS_RANGE;

    if (average < REJECT_THRESHOLD)
        return REJECTED;                    // R3
    if (average >= ACCEPT_THRESHOLD && consensus)
        return ACCEPTED;                    // R2
    return REVISION_REQUIRED;               // R4, R5
}
```

6.4 Diagram-to-Code Traceability

Table 10 maps every forward message call in the optimised sequence diagram (Task 4) to the corresponding Java method invocation, confirming that the code is aligned with the optimised model.

Table 10: Optimised sequence diagram traceability

#	Diagram Message	Java Method Call
1	Researcher → UI : submitResearchOutput(data)	`UI.submitResearchOutput()`
2	UI → Controller : submit(data)	`SubmissionController.submit()`
3	Controller → Validator : validateFormat(data)	`Validator.validateFormat()`

#	Diagram Message	Java Method Call
4	Controller → SubRepo : save(data)	`SubmissionRepository.save()`
5	Controller → RAS : assignAndCollectScores()	`ReviewAssignmentService.assignAndCollectScores()`
6	RAS → ReviewerRepo : findAll()	`ReviewerRepository.findAll()`
7	loop: RAS → Reviewer : generateScore()	`Reviewer.generateScore()`
8	Controller → EvalService : evaluate(id, scores)	`EvaluationService.evaluate()`
9	EvalService → ScoreRepo : saveAll(scores)	`ScoreRepository.saveAll()`
10	EvalService → DecisionEngine : evaluate(scores)	`DecisionEngine.evaluate()`
11	Controller → NotifService : notify(outcome)	`NotificationService.notify()`

6.5 Functional Equivalence

Both implementations produce identical outcomes for the same inputs:

Table 11: Functional equivalence verification

Scenario	Baseline	Optimised
Valid submission (scores 7.5, 8.0, 8.5)	ACCEPTED	ACCEPTED
Invalid submission (empty title)	ERROR	ERROR

7 Task 6 – Empirical Evaluation

7.1 Methodology

Both implementations were benchmarked using identical test scenarios (a valid submission with 5 reviewers, 3 eligible). Measurements include:

1. **Method call counts:** Using a `CallCounter` utility class that increments an atomic counter on each method entry.
2. **Execution time:** 100 warm-up runs (JIT compilation), followed by 1,000 timed runs. Console output is suppressed during timing to avoid I/O bias.
3. **Code complexity metrics:** Lines of code, methods per class, and constructor dependencies counted from source files.

The benchmarks were run on **macOS** with **OpenJDK 21.0.10** (2026-01-20 LTS).

7.2 Method Call Counts

7.2.1 Baseline — 26 Total Method Calls

Table 12: Baseline method call counts for a single valid submission

Method	Count
UI.submitResearchOutput	1
SubmissionController.submit	1
Validator.validateFormat	1
Database.saveSubmission	1
ReviewerManager.getAvailableReviewers	1
Database.fetchReviewers	1
ReviewerManager.filterConflicts	1
ReviewerManager.checkWorkload	1
Reviewer.assignReview	3
EvaluationManager.startEvaluation	1
Reviewer.submitScore	3
EvaluationManager.submitScore	3
Database.saveScore	3
EvaluationManager.calculateAverage	1
EvaluationManager.checkConsensus	1
EvaluationManager.applyRules	1
NotificationService.notifyAcceptance	1
NotificationService.sendNotification	1
TOTAL	**26**

7.2.2 Optimised — 13 Total Method Calls

Table 13: Optimised method call counts for a single valid submission

Method	Count
UI.submitResearchOutput	1
SubmissionController.submit	1
Validator.validateFormat	1
SubmissionRepository.save	1
ReviewAssignmentService.assignAndCollectScores	1
ReviewerRepository.findAll	1
Reviewer.generateScore	3
EvaluationService.evaluate	1
ScoreRepository.saveAll	1
DecisionEngine.evaluate	1
NotificationService.notify	1
TOTAL	**13**

7.2.3 Call Count Analysis

The optimised version reduces method calls by **50%** (26 → 13). The key reductions are shown in Table 14.

Table 14: Sources of method call reduction

Change	Net Saved	Explanation
Batch score persistence	2	3 × `Database.saveScore` → 1 × `ScoreRepository.saveAll`
Reviewer decoupling	3	3 × `Reviewer.submitScore` + 3 × `EvalManager.submitScore` eliminated; `Reviewer.generateScore` already counted
Self-call elimination	2	3 self-calls (`calculateAverage`, `checkConsensus`, `applyRules`) → 1 × `DecisionEngine.evaluate`
Combined reviewer filtering	2	`filterConflicts` + `checkWorkload` inlined into service
Reviewer assignment absorbed	3	3 × `Reviewer.assignReview` absorbed into `assignAndCollectScores`
Combined notification	1	`notifyAcceptance` + `sendNotification` → 1 × `notify(outcome)`
startEvaluation eliminated	0	Replaced by `EvaluationService.evaluate` (1 for 1)
Total saved	**13**	26 − 13 = 13

7.3 Execution Time

Table 15: Execution time comparison (1,000 runs after 100 warm-up)

Metric	Baseline	Optimised	Change
Mean	26,502 ns (0.027 ms)	24,223 ns (0.024 ms)	−9%
Median	20,000 ns (0.020 ms)	16,563 ns (0.017 ms)	−17%
Min	12,167 ns	8,459 ns	−30%
Max	229,125 ns	294,542 ns	—
StdDev	21,552 ns	22,968 ns	—

Analysis: The optimised version is faster across all central-tendency metrics (mean, median, and minimum). This is attributable to:

1. **50% fewer method calls:** 13 calls vs 26 means fewer stack frames, less dispatch overhead, and less `CallCounter` instrumentation.

2. **Batch persistence:** A single `saveAll()` call replaces three individual `saveScore()` calls, reducing HashMap operations.
3. **Elimination of self-calls:** The five self-calls in the baseline each carry method invocation overhead that the optimised version avoids.
4. **Streamlined notification:** One `notify(outcome)` call replaces a switch statement dispatching to three separate methods plus a `sendNotification()` call.

The maximum time is higher for the optimised version (294,542 ns vs 229,125 ns), likely due to garbage collection pauses from the additional object allocations (3 repositories vs 1 Database). At the nanosecond scale, outliers are dominated by JIT recompilation and GC. Both versions execute in under 0.03 ms in the median case — the difference of ∼3,400 ns (0.003 ms) is imperceptible in any real-world context.

7.4 Code Complexity Metrics

7.4.1 Lines of Code per Core Class

Table 16: Lines of code comparison (core classes only)

Baseline Class	**LOC**		**Optimised Class**	**LOC**
SubmissionController	95	→	SubmissionController	65
EvaluationManager	133	→	EvaluationService	39
ReviewerManager	80	→	ReviewAssignmentService	69
Database	66	→	SubmissionRepository	26
Reviewer	66	→	ReviewerRepository	23
NotificationService	53		ScoreRepository	30
Validator	43		DecisionEngine	57
UI	33		NotificationService	19
			EvaluationOutcome	11
			Validator	37
			Reviewer	44
			UI	21
Total	**569**		**Total**	**441**
Average	**71**		**Average**	**37**

The optimised version has **22% fewer total lines** despite having more classes. Average class size drops from 71 LOC (8 classes) to 37 LOC (12 classes), indicating better cohesion.

7.4.2 Constructor Dependencies (Efferent Coupling)

Table 17: Constructor dependency comparison

Class	Baseline	Optimised
SubmissionController	4	5 (focused services)
ReviewerManager / ReviewAssignmentService	1	1
EvaluationManager / EvaluationService	2 (Database, NotificationService)	2 (ScoreRepository, DecisionEngine)
NotificationService	0	0

7.4.3 Maintainability Scenarios

Table 18: Impact analysis for common change scenarios

Change Scenario	Baseline	Optimised
Add new evaluation outcome	3 classes	2 classes
Change score persistence strategy	2 classes	1 class
Add new reviewer eligibility criterion	1 class	1 class
Add audit logging for decisions	1 class	1 class

7.5 Evaluation Summary

Table 19: Overall empirical evaluation summary

Metric	Baseline	Optimised	Improvement
Method calls per submission	26	13	−50%
Total LOC (core classes)	569	441	−22%
Average LOC per class	71	37	−48%
Number of classes	8	12	+4 (trade-off)
Max class LOC	133	69	−48%
Self-calls in sequence	5	0	**Eliminated**
Database calls per submission	5	3	−40%
Classes changed for new outcome	3	2	−33%
Execution time (median)	0.020 ms	0.017 ms	−17%

The optimised design delivers substantial improvements across all measured metrics: method call efficiency (−50%), code size (−22%), average class size (−48%), execution time (−17% median), and maintainability. The only trade-off is an increased number of classes (8 → 12), which is a deliberate architectural decision to improve cohesion and separation of concerns.

7.6 Trade-offs Introduced by Optimisation

1. **Increased structural complexity:** The optimised design has 12 classes vs 8. A developer new to the codebase must understand more files, though each file is smaller and more focused. Navigation overhead increases, but comprehension per file decreases.
2. **Higher object allocation:** Each submission run instantiates 3 repository objects, 3 services, a DecisionEngine, and an enum value, versus 1 Database, 1 ReviewerManager, and 1 EvaluationManager. This marginally increases GC pressure, visible in the higher maximum execution time (294,542 ns vs 229,125 ns).
3. **SubmissionController dependency count increased:** The controller went from 4 to 5 constructor dependencies. While each dependency is now a focused single-responsibility service, the raw dependency count is higher. This could be mitigated by introducing a façade if further services are added.
4. **Indirection cost:** The `EvaluationService` → `DecisionEngine` delegation adds a layer of indirection. A developer tracing the evaluation flow must follow one additional hop. However, this enables independent testing and evolution of the decision logic.

These trade-offs are characteristic of refactoring towards higher cohesion: structural complexity increases to reduce behavioural complexity. The empirical data confirms that the net effect is positive across all measured dimensions.

8 Conclusion

This assignment demonstrated the full lifecycle of design optimisation: from faithfully implementing a baseline behavioural model, through systematic design analysis, decision table extraction, and architectural redesign, to empirical validation of the improvements.

The baseline implementation exposed several significant design flaws: a fat controller with five responsibilities, a God Object database with three unrelated concerns, redundant notification methods, per-score database persistence, and opaque decision logic buried in self-calls. The decision table (Task 3) made the system's acceptance/rejection/revision logic explicit and traceable.

The optimised implementation addressed every identified issue through the Repository pattern (splitting the monolithic Database), service extraction (separating reviewer assignment and evaluation into dedicated services), a centralised DecisionEngine (encapsulating the decision table), and parameterised notifications. Empirical evaluation confirmed a **50% reduction in method calls**, **22% reduction in total code**, **48% reduction in average class size**, and a **17% faster median execution time**.

The trade-off of having more classes (12 vs 8) is justified by the dramatically lower per-class complexity and improved maintainability — demonstrated by the reduced number of classes that need modification when adding new features or changing existing behaviour.